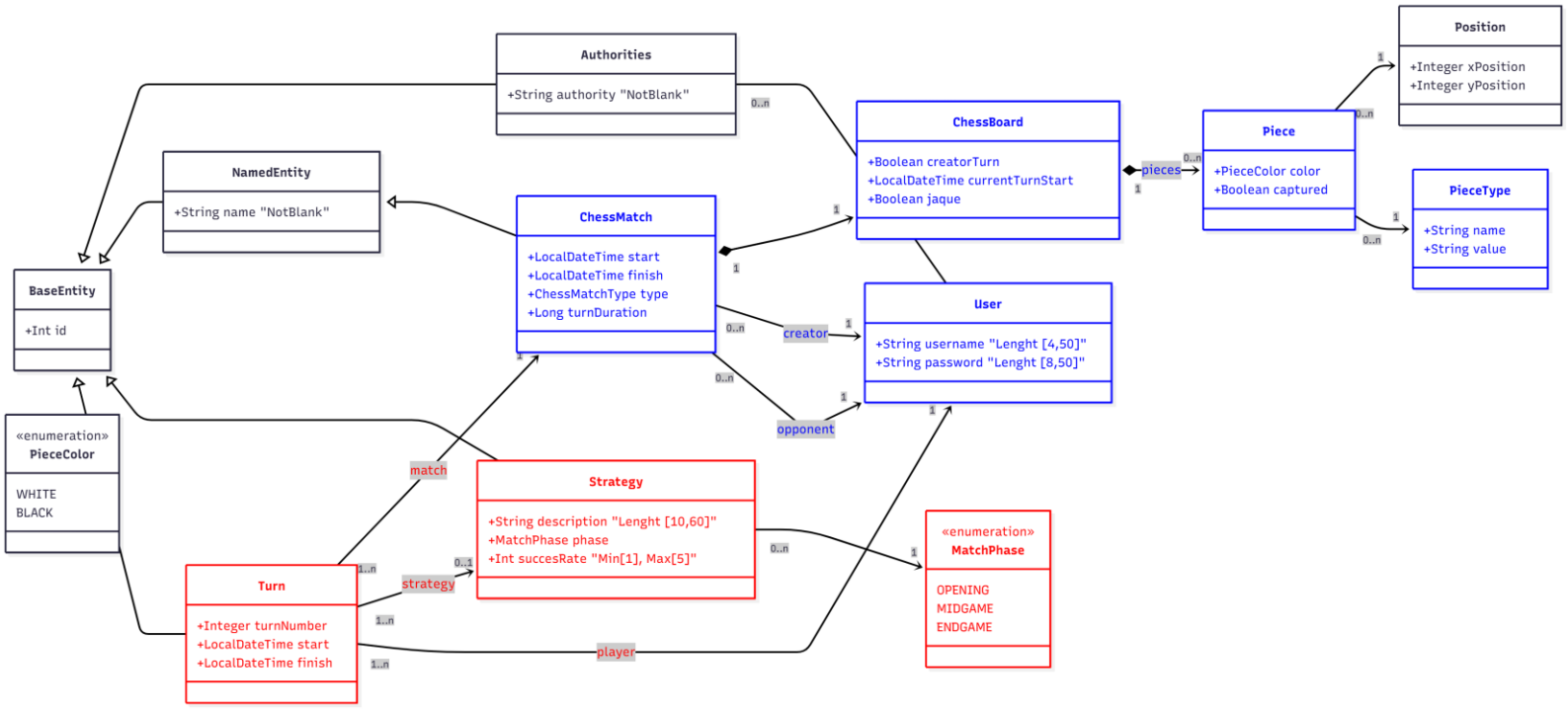
Lab control-check of DP1 2025-2026 (First Call January 2026)

Statement

In this exercise, we will add the functionality of a strategy log for chess games. Specifically, we provide a "ChessMatch" class that represents the games played, and which has an associated instance of the "ChessBoard" class that represents the board state for that game, and therefore will have an associated set of instances of the "Piece" class. Additionally, we have the " Strategy " class , which represents a strategy adopted by a player on a specific turn of a game, for example, "play the Italian Opening on the first turn," or "attack the center on turn 15." To express the phase of the game in which the strategy is applied, we use the " MatchPhase " enumeration. The players in a game record the strategy they will use.

The UML diagram that describes the classes and relationships we will be working with is as follows:



The classes for which we will perform object-relational mapping as JPA entities are highlighted in red. The classes in blue are classes that are provided already mapped, but which will be used during the lab exercise.

We will perform a series of exercises based on functionalities that we will implement in the system and validate using unit tests. If you wish to see the results of the backend tests , you can run them (either using your preferred development environment or by using the command " *mvnw test* " in the project's root folder). Each correctly completed exercise will be worth one point. The number of test cases in each exercise may vary, and the grade will be calculated based on the percentage of test cases that pass. For example, if half (50%) of the test cases in an exercise pass, you will receive 0.5 points instead of one.

To begin the assessment, you must accept the assignment for this practical assessment via the link provided in the EV platform.

By accepting this task, a unique repository will be created for you. You must use this repository to complete the practical assignment. You must submit the EV activity associated with the assignment, providing the URL of your personal repository as the text. Remember that you must also submit your solution to the assignment.

The delivery of your solution to the control system will be done using a single command: "*git" Push* the project to your individual repository. Remember to push before logging off your computer and leaving the classroom; otherwise, your attempt will be marked as not submitted. Your first task for this assignment is to clone the project (remember that if you are using the classroom computers for the assignment, you will need to use a GitHub authentication token as your key; you have a help document for setting it up in the assignment's repository). Next, you will need to import the project into your preferred development environment and begin the exercises listed below. When you import the project, it may contain compilation errors. Don't worry; if any exist, these errors will disappear as you complete the different exercises for the assignment.

Important Note 1: Do not modify the class names or the signature (name, response type, and parameters) of the methods provided as background material for the assessment. The tests used for evaluation depend on the classes and methods having the provided structure and names. If you modify them, you will likely fail the tests and receive a poor grade.

Important Note 2: Do not modify the unit tests provided as part of the project under any circumstances. Even if you modify the tests in your local copy of the project, they will be restored using a git command before the tests are run for the final grade, so your modifications to the tests will not be considered at any time.

Important note 3: As long as there are unsolved exercises, there will be tests that do not work, and therefore the command " *mvnw" will not work. The ` install` command* will end with an error. This is normal due to the way the control is implemented and is nothing to worry about. If you want to test the application, you can run it as usual even though ` mvnw` will fail. install " ends with error.

Important Note 4 : Downloading the test material using Git , and submitting your solution with Git through the GitHub repository created for this purpose are part of the skills assessed during the exam, therefore submissions that do not use this method will not be accepted, and you will not be able to ask the teachers for help to carry out these tasks.

Important Note 5 : Projects whose source code does not compile correctly or that cause application startup failures during Spring context initialization will not be accepted as valid solutions. Solutions whose source code does not compile or that are unable to start the Spring context will receive a score of 0.

Test 1 – Creation of the Strategy and Turn entities and their associated repositories

Modify the "Strategy" class to make it an entity. This class is in the " es.us.dp1.chess.tournament.strategy" package and must have the following attributes and constraints:

- The integer attribute named "id" will act as the primary key in the relational database table associated with the entity.
- A string attribute named "description" is mandatory (it cannot be null), which must have a minimum length of **10** characters and a maximum length of **60** .[1]
- An attribute of the enumerated type "MatchPhase" named "phase " indicates the phase the game is in when the strategy is recorded. This attribute is mandatory and must be stored as a string in the database ("OPENING", "MIDGAME", or "ENDGAME").
- The integer attribute named "successRate ", which represents the degree of success achieved with the strategy and must take a value between 1 and 5. This attribute will be specified after executing the strategy and is therefore an optional attribute that will not have any associated value while the strategy is running.

Modify the "Turn" class to make it an entity. This class is located in the " es.us.dp1.chess.tournament.match " package and must have the following attributes and constraints:

- The integer attribute named "id" will act as the primary key in the relational database table associated with the entity.
- mandatory integer attribute named "turnNumber" will indicate the turn number of the game, being an incremental number regardless of the player playing, so that the creator of the game will play first on turn 1, and the opponent will play for the first time on turn 2. Therefore, the minimum allowed turn number will be 1.
- The time instant attribute (LocalDateTime) named "startTime", which represents the instant when the player's turn begins, is a mandatory attribute.
- The time instant attribute (LocalDateTime) named "finishTime", which represents the instant at which the player's turn ends and is an optional attribute since while the player is on their turn it will not have any associated value.

Do not modify the @Transient annotations of the classes for now. Modify the "StrategyRepository" and "TurnRepository " interfaces, hosted in the same packages as "Strategy" and "Turn" respectively, to extend CrudRepository . Don't forget to specify their type parameters.

Test 2 – Creating relationships between entities

Remove the @Transient annotations from the methods and attributes that have them in the entities created in the previous exercise. Create the following relationships between the entities: Create a one-way relationship from "Turn" to "ChessMatch" that reflects the relationship shown in the UML diagram (on the first page of this document), using the "match" attribute. Ensure that the relationships accurately reflect the cardinality shown in the UML diagram.

Create a one-way relationship from "Turn" to " Strategy " that reflects the relationship shown in the UML diagram (on the first page of this statement), using the "strategy " attribute. Ensure that the relationships

[1]Note that these restrictions do not match those of NamedEntity.

accurately reflect the cardinality shown in the UML diagram, considering that attributes can be null since the cardinality at the navigable end is 0..1 .

Create a one-way relationship from “Turn” to “User” that reflects the relationship shown in the UML diagram (on the first page of this statement), using the “player” attribute. You must ensure that the relationships accurately reflect the cardinality shown in the UML diagram.

Test 3 – Modification of the database initialization script to include three strategies and two turns

Modify the database initialization script to create the following strategies:

Strategy 1:

- id: 1
- description: Focus on rapid development and control of the center
- phase : OPENING
- successRate : null

Strategy 2:

- id: 2
- description: Prioritize piece safety and solid positioning
- phase : MIDGAME
- successRate : null

Strategy 3:

- id: 3
- description: Aim for king safety and pawn structure integrity
- phase : ENDGAME
- successRate : null

Turn 1:

- id: 1
- turnNumber : 1
- startTime : 2025-10-20 12:00
- endTime : 2025-10-20 12:01

Turn 2:

- id: 2
- turnNumber : 2
- startTime : 2025-10-20 12:01
- endTime : 2025-10-20 12:02

Test 4 – Modification of the database initialization script to link turns with strategies, games, and players

Modify this database initialization script so that:

- The shift whose id is 1 is associated with:
    - the strategy whose id is 1,
    - the game whose ID is 15, and
    - the user whose id is 4.
- The shift whose id is 2 is associated with:
    - no strategy,
    - the game whose ID is 15, and
    - the user whose id is 5.

Note that the order in which the INSERT statements appear in the database initialization script is relevant when defining associations.

Test 5 – Creation of strategy management services

Modify the " TurnService " class so that it becomes a Spring business logic service and provides an implementation of the methods that allows:

1. Get all shifts stored in the database as a list using the repository (findAll method).
2. Get a turn by Id (findById method).
3. Save a turn to the database, ensuring that the user assigned the turn is the one currently logged into the system and that this user is the one assigned for the current turn in the game. If the assigned user is not the correct one for that turn, a WrongUserAssignmentException will be thrown (you can use the getCurrentTurn method of the turn repository to obtain the current turn and check the correct user).

All these methods **must be transactional** , but the associated annotations should be done at the method level, not the class level **.** Do not modify the implementation of the other service methods for now.

Test 6 – Annotate the game repository with a complex query

Modify the custom query that can be invoked through the " findCapturePoints " method of the " MatchRepository " game repository (located in the es.us.dp1.chess.tournament.match package), which takes a " ChessMatch " id as a parameter, to return a list of pairs (Object [ ]) containing a color and an integer. The goal is to calculate the **total points of the captured pieces of each color** in that game.

In other words:

- **BLACK** is passed , the method should return the sum of the values of all the **black pieces** captured in that game (i.e., the points obtained by the white player).
- **WHITE** is passed , it must return the sum of the values of the captured **white pieces** (the black player's points).

Note that PieceType has a value attribute that represents the points worth of that piece. To implement this exercise, you must modify the query associated with the repository method named " findCapturePoints ", without modifying its signature (response type, name, and parameters).

Test 7 – Creating a controller to return captured pieces to the opponent by a player in an available game

Create a method in "*ChessMatchController "* (located in the es.us.dp1.chess.tournament.match package) *The method* should return the pieces that a logged-in player has captured from their opponent in a game. It must respond to GET requests at the following URL:

http://localhost:8080/api/v1/matches/X/captures/

The method should return the screenshots of the user logged into the system for a specific game, where X is the game ID. Remember that the UserService class has a " findCurrentUser ()" method that allows you to obtain the currently logged-in user; you must use this method to get the current user.

The status code 404 (NOT_FOUND) should be returned in the following cases:

- If there is no item with the specified ID in the system.
- In case no pieces have been captured yet.

If the logged-in user is not one of the players in that game, a 403 (FORBIDDEN) status code should be returned. Note that predefined exceptions are provided to represent these scenarios, such as the ResourceNotFoundException or AccessDeniedException classes .

To implement this exercise, you must modify both the ChessMatchService class (getCaptures method) and the ChessMatchController class (adding a method to the controller to respond to requests). Implement the business logic and communication management where appropriate.

Test 8 – Implement a test for an algorithm that checks the state of the game for a bishop move (" bishop ") (specifically if there is check).

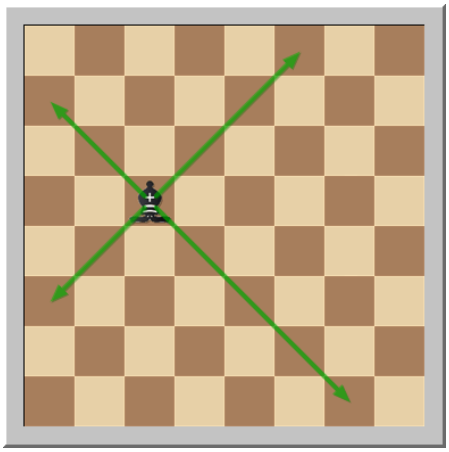
To facilitate interaction with the page, a set of algorithms has been created to check if our king is in check. In this exercise, we will create a set of test cases to verify that the algorithm works correctly.

The algorithm's interface is specified in the " BishopCheckDetector " interface found in the " es.us.dp1.chess.tournament.match.checkdetector " package, which aims to detect when an opposing bishop is in check to our king (assuming no pieces are blocking the path). Four implementations of the algorithm are provided. Specifically, the method that checks the check state has the following signature: boolean isJaque (int bishopX , int bishopY , int kingX , int kingY). If the algorithm is passed positions that are invalid by definition (note that the values of all parameters should always be between 0 and 7 to represent valid positions on the board), the algorithm should throw an InvalidPositionException .

Modify the test class named " BishopJaqueDetectorTest " located in the folder " scr /test/java/es/ us /dp1/ chess / tournament / bishopjaquedetector " and specify as many methods with test cases as you deem necessary to validate the algorithm's correct operation. Note that the class has an attribute of type BishopJaqueDetector named algorithm ; use this attribute as the subject under test in all your test methods.

Their implementation of the test **You must not use mocks , nor spring test annotations** (@DataJpaTest, @SpringBootTest , etc.), **nor parameterized tests, and all methods annotated with @Test must be parameterless** .

Remember that the bishop's movement in chess has the following structure:



Test 9 – Creating a basic React component from information about tiles captured on a board

Modify the React component provided in the file “frontend/ src / matches /board.js” to display the capture details on the board for the current player. To do this, you must use the API by sending a GET request to the URL [api/v1/ matches /X/ captures]to obtain the list of pieces captured by the logged-in player. This API returns a result similar to the following:

```
[
  { " id" : 5000 ,
    "position" : null ,
    "type " : {
      "name" : "QUEEN" ,
      "value" : 9
},
    "color" : "BLACK" ,
    "captured" : true } ,
{
    "id" : 5001 ,
    "position" : null ,
    "type " : {
      "name" : "ROOK" ,
      "value" : 5
},
    "color" : "BLACK" ,
    "captured" : true
}
]
```

This component should display the following dashboard data:

- A level 1 title whose content is the text “Board info ”
- A level 3 title whose content is “Captures”

- Display the total points of the captures obtained in a div . For example , in the case shown above, it should display the text: “Total Points: 14”.

To run this test and check its result, navigate to the frontend folder and execute the command `npm test`, then press 'a' in the Jest command menu . Note that you must have previously run the `npm` command at least once. install to ensure all node libraries are installed.

Test 10 – Creation of a component to display on screen in a game the pieces captured from the opposing player.

Complete the code for the boardDetail component to display an unordered list ( ) of the captured tiles. Specifically, each captured tile should be displayed in a list item ( ), showing the tile's color, a hyphen , and the tile type name. For example, the response shown in the previous exercise would be displayed as follows:

- BLACK-QUEEN
- BLACK-ROOK

To run this test and check its result, navigate to the frontend folder and execute the command `npm test`, then press 'a' in the Jest command menu . Note that you must have previously run the `npm` command at least once. install to ensure all node libraries are installed.